

Lab 11 - Agent evaluation scorecard

AI Agents Fundamentals for Beginners | C690 | v1.0

Objective

Compute task-quality and cost metrics with explicit denominators.

Alignment: LO3. Scheduled hands-on time: 35 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- predictions.csv
- runs.csv

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Inspect predictions.csv. Identify true_label and predicted_label. Inspect runs.csv for success, processing_minutes, review_minutes and cost_sgd.
2. Run python3 run.py. Independently check TP=18, FN=2, FP=6 and TN=74; precision should be 0.75 and recall 0.90.
3. Calculate cost per successful task from all run costs divided by accepted successes, including failed-run costs in the numerator.
4. Use prompt 1 to produce a pilot scorecard. Require the raw counts, denominators, limitations and a separate quality floor.
5. Use prompt 2 to challenge an accuracy-only success claim. Explain why missed urgent cases matter despite 92-percent accuracy.
6. Save a release recommendation that names the owner, unresolved failure cases and next experiment. Do not present synthetic results as a vendor benchmark.

Acceptance checks

- Precision=0.75, recall=0.90, accuracy=0.92.
- Cost per success includes the cost of failed runs.
- Release recommendation includes failure evidence, not only averages.

Run python verify.py after run.py (use python3 if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's requirements.txt.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.

- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after the lab; keep source data and prompts unchanged.